

Ch. 8

# 邏輯運算與條件分支

Horazon

應用程式設計

# 本章目標

1. 學習 **邏輯運算** (Logic Operators)：And, Or, Not。
2. 學習 **巢狀邏輯** 與 **真值表** (Truth Table)。
3. 製作「剪刀石頭布」遊戲。
4. 體驗 **演算法設計** 的思維。

# 為什麼需要邏輯運算？

之前的 If 都只有一個條件：**如果 分數 > 60**。

但真實世界往往更複雜：

- **遊樂園門票**：身高 < 120 **或者 (OR)** 年齡 < 6 歲 -> 免費。
- **登入系統**：帳號正確 **而且 (AND)** 密碼正確 -> 登入成功。
- **請假**：**不是 (NOT)** 假日 -> 就要上課。

這些 **AND** , **OR** , **NOT** 就是我們今天要學的 **邏輯運算子**。

# 運算子介紹：AND, OR, NOT

在 Logic (邏輯) 抽屜 (綠色) 裡：

## 1. AND (且)

- 兩個條件 **都要成立** 才是 True。
- 嚴格的守門員。

## 2. OR (或)

- 兩個條件 **只要一個成立** 就是 True。
- 寬鬆的守門員。

## 3. NOT (非)

- 唱反調的。
- True 變 False，False 變 True。

# 真值表 (Truth Table)

這張表工程師都要背下來：

假设 P 和 Q 是两个判断条件：

| P | Q | P AND Q | P OR Q |
|---|---|---------|--------|
| T | T | T       | T      |
| T | F | F       | T      |
| F | T | F       | T      |
| F | F | F       | F      |

AND: 全真才真。

# 專案實作：剪刀石頭布

這是一個經典的邏輯練習題。

## 難點在哪？

1. **電腦怎麼出拳？** -> 轉化為數字與亂數問題。
2. **怎麼判斷輸贏？** -> 電腦 3 種拳 x 玩家 3 種拳 = 9 種組合。
  - 要寫 9 個 If 嗎？
  - 還是有更聰明的寫法？

# 步驟 1：數字模型化 (Modeling)

電腦看不懂圖片，我們先把「拳」變成「數字」：

- 1: 剪刀
- 2: 石頭
- 3: 布

這樣電腦只要隨機選 1~3，就是隨機出拳了！

## 變數設定

- `computer_move` (變數)：存電腦出的 (1, 2, 3)。
- `player_move` (變數)：存玩家出的 (1, 2, 3)。

## 步驟 2：畫面配置

1. **電腦出拳區**：放一張 Image，一開始顯示問號，之後根據數字換圖 (p1.png, p2.png, p3.png)。
2. **玩家出拳區**：三個**按鈕**，分別是剪刀、石頭、布。
3. **結果區**：Label 顯示「平手」、「贏」、「輸」。

圖片命名的小技巧：`p1.png` 代表剪刀。

這樣程式只要 `Set Image.Picture to join ("p", computer_move, ".png")` 就可以一句話換圖！



# 步驟 3：輸贏判斷演算法 (Algorithm)

## 方法 A：暴力法 (巢狀 If)

```
如果 玩家 = 1 (剪刀)  
    如果 電腦 = 1 -> 平手  
    如果 電腦 = 2 -> 輸  
    如果 電腦 = 3 -> 贏  
否則如果 玩家 = 2 (石頭)  
    ... (再寫3個)  
否則如果 玩家 = 3 (布)  
    ... (再寫3個)
```

這太累了，程式碼又長又醜。

# 步驟 3：輸贏判斷演算法 (Algorithm)

## 方法 B：邏輯歸納法 (Logic)

我們先把規則分類：

1. **平手**：最簡單， **玩家 = 電腦** 。
2. **玩家贏**：有三種情況 (用 OR 連接) 。
  - (玩家=1 AND 電腦=3) **OR**
  - (玩家=2 AND 電腦=1) **OR**
  - (玩家=3 AND 電腦=2)
3. **玩家輸**：剩下都是輸 (Else) 。

這樣只要寫 3 個大的 If 區塊！

# 實作：邏輯積木

```
當 玩家出拳 (player_move)
  呼叫 電腦出拳 (產生 random computer_move)

  如果 (player_move = computer_move)
    顯示 "平手"
  否則如果 ( (player_move=1 AND computer_move=3) OR
              (player_move=2 AND computer_move=1) OR
              (player_move=3 AND computer_move=2) )
    顯示 "你贏了"
  否則
    顯示 "你輸了"
```

這在拼積木時，需要一點耐心把 AND 和 OR 組合起來。

# 方法 C：數學解法 (高手進階)

其實剪刀石頭布有一個數學規律：

- 贏的情況： $(\text{玩家} - \text{電腦}) = -2$  或  $1$
- 輸的情況： $(\text{玩家} - \text{電腦}) = -1$  或  $2$

我們可以只用數學算式算出輸贏！

(這是運算思維的高階展現，有興趣的同學可以試試看)

# 複雜邏輯：巢狀與優先權

當邏輯變複雜時： `A AND B OR C`

電腦會先算哪個？

- 就像數學的先乘除後加減，邏輯也有優先權。
- 但在 AI2 積木裡，因為是**拼圖**，所以層級看得很清楚。
- 被包在里面的先算。

建議多用括號 (或是積木的層級) 讓邏輯清楚，不要讓別人猜。

# 練習：三戰兩勝制

試著加入變數 `player_wins` 和 `computer_wins` 。

1. 每次判斷完輸贏，就更新分數。
2. 判斷 如果 `player_wins >= 2` -> 顯示總冠軍畫面。
3. 提供 Reset 按鈕，把變數歸零。

# 重點回顧

1. **AND (且)**: 嚴格，全對才對。
2. **OR (或)**: 寬鬆，其一對就對。
3. **NOT (非)**: 真假反轉。
4. **Modeling**: 把剪刀石頭布變成 1, 2, 3。
5. **Algorithm**: 找出判斷輸贏最簡潔的邏輯。

這就是程式設計師的腦袋運作方式！